

ONBOARDING · DESTRAVE CLAUDE CODE

O kit que faz o Claude Code virar *coluna vertebral*

Um repositório, um comando, dez minutos. Você clona o destrave-starter-kit, roda `bash install.sh` (ou `install.ps1` no Windows pra subir o WSL primeiro), e em poucos minutos seu ambiente vira o meu: terminal ZSH com Oh My Zsh, cinco skills core funcionando e o ritual de fim de sessão instalado. Sem decoreba, sem copia-cola de prompt.

O QUE ENTRA · FASE 0 + 5 SKILLS CORE

ITEM	CATEGORIA	PRA QUÊ
Terminal	Fase 0 · ambiente	ZSH + Oh My Zsh + plugins (autosuggestions, syntax-highlighting)
RTK	Dev essencial	Economia de 60-90% de token em operações repetidas
Dictation	Dev essencial	Falar com o Claude em vez de digitar (Whispering, cross-platform)
agent-browser	Dev essencial	CLI de navegação web pro agente abrir páginas e ler conteúdo
Wiki	Captura/memória	Segundo cérebro persistente em markdown, navegável no Obsidian
/done	Ritual de fim	Fechar etapa, calibrar skills (auto-improvement) e bridge ClickUp opcional

Terminal — o lugar onde você vai *viver*

Antes das skills, o terminal. Razão simples: você vai passar muitas horas dentro dele. Um terminal lento, sem autocomplete e sem syntax highlight é fricção que se paga em todas as sessões seguintes. **ZSH + Oh My Zsh** + dois plugins essenciais (autosuggestions, syntax-highlighting) é a stack que eu uso todo dia. No Windows, isso roda dentro do WSL2 Ubuntu — não tem versão "PowerShell" porque ZSH é Unix.

TIPO	PRÉ-REQUISITOS	QUANDO NÃO USAR
Shell + framework + plugins	Mac/Linux: nativo · Windows: WSL2 (install.ps1 instala)	Nunca. O terminal é o solo de tudo.

COMO INSTALA

- Roda dentro do `install.sh` como Fase 0 (opção `0` ou parte do `a`).
- Instala `zsh` (se faltar), Oh My Zsh, e clona os 2 plugins em `~/.oh-my-zsh/custom/plugins/`.
- Patcha seu `.zshrc` ativando os plugins (idempotente, backup em `.zshrc.destrave-backup`).
- Torna `zsh` o shell padrão. Abra um terminal novo pra ver.

Atalhos top: ↑ histórico fuzzy · → aceita a sugestão cinza · Tab autocomplete · Ctrl+R busca reversa · cd – volta diretório · !! repete último comando (clássico: sudo !!).

RTK — economia *silenciosa* de token

Proxy CLI escrito em Rust que intercepta comandos do Claude Code via hook e filtra output verboso antes de chegar no modelo. Você não muda nada na forma de trabalhar — só gasta 60-90% menos token em operações repetidas (git status, ls, find, etc.).

TIPO	PRÉ-REQUISITOS	QUANDO NÃO USAR
CLI binário + hook Claude Code	Mac: Homebrew · Linux/WSL: cargo (Rust)	Debug profundo de comando — use rtk proxy <cmd>

COMO INSTALA

- Mac: `brew install rtk-ai/rtk/rtk` (o installer faz por você).
- Linux/WSL: `cargo install rtk-cli` (installer instala rustup se faltar).
- Copia o hook `rtk-rewrite.sh` pra `~/.claude/hooks/` e patcha `~/.claude/settings.json` via jq — idempotente.
- Em sessão Claude Code, comandos Bash são reescritos automaticamente. Confira economia com `rtk gain`.

Verificação visual: depois de uns dias usando, rode `rtk gain` e veja a economia acumulada. Em sessões longas, é comum bater 70%+ em comandos de dev.

Whispering — você fala, o *Claude* entende

Dictation cross-platform (open-source). Você segura uma hotkey, fala normalmente, solta — o texto aparece onde o cursor estiver. Plug direto no chat do Claude Code. Suporta Whisper local (privado) ou OpenAI API (mais rápido). Substitui o OpenWispr da v1 porque funciona em Mac, Linux e Windows.

TIPO	PRÉ-REQUISITOS	QUANDO NÃO USAR
App Electron (Mac/Win/Linux)	Mic + ~2GB livres se for usar Whisper local	Audio com música/barulho — qualidade cai

COMO INSTALA

- Mac: `brew install --cask whispering` (installer faz).
- Linux: Flatpak ou Applmage (installer tenta; cai pra link manual se faltar).
- Windows: instala o `.exe` no host (não dentro do WSL — mic + cursor moram no Windows). Link no README do skill.
- Na primeira abertura: dá permissão de mic + acessibilidade, define hotkey (padrão F10) e escolhe backend (OpenAI ou Whisper local).

Por que importa: a maior parte do meu input pro Claude vem por voz. Sessão de 2h tem facilmente 30+ ditados longos. Sem isso, a fricção mata o uso.

agent-browser — o Claude com navegador próprio

CLI da Vercel Labs que dá ao Claude um browser headless com API simples. Diferente de WebFetch (que só puxa HTML), o agent-browser navega de verdade: clica, preenche form, lê screenshot. Refs @eN tornam a árvore amigável pro modelo. É a primeira escolha pra qualquer scraping ou navegação assistida.

TIPO	PRÉ-REQUISITOS	QUANDO NÃO USAR
CLI global Node.js	Node 20+ · npm (installer cuida)	API pública disponível — usa direto

COMO INSTALA

- Installer garante Node 20+ (brew / NodeSource / dnf / pacman).
- `npm install -g @vercel-labs/agent-browser`.
- Registra regra de hierarquia no `~/.claude/CLAUDE.md` global: `API > agent-browser > playwright > puppeteer`.
- Teste: `agent-browser snapshot https://example.com` — devolve árvore com refs `@eN`.

Caso real: "abre o dashboard X, copia o número de leads dessa semana e cola no relatório." Sem agent-browser, isso é manual ou exige scraping custom. Com ele, é uma instrução conversacional.

Wiki — seu segundo *cérebro* versionado

Não é uma skill instalável "tradicional" — é um padrão. Uma pasta `wiki/` no seu repo, mantida pelo próprio Claude, que vira fonte de verdade pra clientes, padrões e decisões. Navegável no Obsidian (grafo, backlinks, callouts) e indexável pelo modelo. O kit já vem com o template enxuto: `_schema.md`, `index.md`, `log.md` e as 5 subpastas mínimas.

TIPO	PRÉ-REQUISITOS	QUANDO NÃO USAR
Convenção de pastas + template markdown	Git repo · Obsidian (opcional, recomendado)	Fragmentos soltos / hipóteses datadas — vão pro MemPalace

COMO INSTALA

- Installer pergunta onde criar a wiki (default: pasta atual).
- Copia template enxuto: `_schema.md`, `index.md`, `log.md` + subpastas `clientes/` `padroes/` `decisooes/` `projetos/` `_fontes/`.
- Não cria páginas automaticamente — espera você ingerir conhecimento real (calls, docs).
- Sugere apontar Obsidian pra essa pasta pra navegar o grafo.

Cuidado: wiki só funciona se você ingerir conhecimento de verdade. Página vazia não ajuda. A regra que eu sigo: depois de toda call ou doc novo, pergunto "isso vira wiki?". Maioria não — mas as que viram, valem ouro 6 meses depois.

/done — fechar etapa, *calibrar* skill, comemorar

O ritual de fim de toda etapa. Faz três coisas numa passada: registra o que foi entregue, varre as skills usadas na sessão perguntando se alguma precisa ajuste (auto-improvement Nível 4 — a skill aprende com você) e, se você usa ClickUp, fecha a task. Quem roda /done regular tem skills que evoluem sozinhas.

TIPO	PRÉ-REQUISITOS	QUANDO NÃO USAR
Slash command (SKILL.md)	Nenhum · ClickUp opt-in via config JSON	No meio da execução — só no fim

COMO INSTALA

- Installer copia `SKILL.md` pra `~/.claude/skills/done/`.
- Se o arquivo já tem calibrações suas, preserva (não sobrescreve).
- Pra ativar ClickUp, crie `~/.claude/state/clickup-config.json` com token + IDs (instruções no SKILL.md).
- Em sessão Claude Code, digite `/done` quando terminar etapa, task ou projeto.

Por que é o último: as outras skills te dão capacidade. /done faz elas *evoluírem* — cada calibração vira regra viva, sua versão pessoal da skill.

MemPalace — memória *semântica* (quando você sentir falta)

MCP server que combina Knowledge Graph (triples sobre entidades nomeadas) e busca vetorial. O Claude recupera percepções que você teve em sessões anteriores, mesmo que nem lembre de ter falado. **Fora do master por design.** Comece com a Wiki — ela resolve 80% do problema com 5% da complexidade. Volte aqui quando sentir falta de "lembrar do que conversamos mês passado".

TIPO	PRÉ-REQUISITOS	QUANDO NÃO USAR
MCP server (pip package)	Python 3.10+ · ChromaDB (~500MB)	Decisão já consolidada — vai pra wiki/decisoes/

COMO INSTALA (OPT-IN)

- Rode `bash skills/mempalace/install-mac-linux.sh` manualmente.
- Installer instala o pacote pip `mempalace` e imprime o snippet pra colar em `.mcp.json`.
- Oferece gravar automaticamente no `.mcp.json` do projeto se você quiser.
- Reinicie Claude Code — `mempalace_status` aparece nas tools.

Armadilha: nunca rode `mempalace mine .` na raiz de um repo grande. Gera milhares de chunks lixo e afoga a busca. Sempre alvo específico (ex: só wiki/).

PRÓXIMO PASSO · MENTORIA

Depois de instalar, *abra uma sessão* comigo

Antes da nossa próxima call, valide o setup com o checklist abaixo. Se algum item travar, manda print na thread da mentoria — a gente desbloqueia ali mesmo.

CHECKLIST DE VALIDAÇÃO

- 01 Terminal:** abra um terminal novo — prompt mudou (zsh ativo) e `echo $SHELL` mostra `/bin/zsh` ou similar. Sugestões cinza aparecem após uns comandos.
- 02 RTK:** `rtk --version` mostra `>= 0.23` e `rtk gain` abre sem erro.
- 03 Dictation:** abra o Whispering, dê permissão de mic, segure a hotkey, fale uma frase, texto aparece no editor ativo.
- 04 agent-browser:** `agent-browser snapshot https://example.com` retorna árvore com refs `@eN`.
- 05 Wiki:** pasta `wiki/` existe no destino escolhido com subpastas mínimas e `_schema.md` preenchido.
- 06 /done:** `~/.claude/skills/done/SKILL.md` existe; em sessão, `/done` abre o ritual.

Esse kit não te transforma em mim. Te dá o mesmo solo de onde eu construo. O que cresce nele é seu.

— G. P.